Introduction to Complexity Theory

Complexity Classes

Time Complexity of an Algorithm

- Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.
- Asymptotic notations are mathematical notations used to represent the time complexity of algorithms for Asymptotic analysis.
- Following are the commonly used asymptotic notations to calculate the running time complexity of an algorithm.
  1. O Notation
  2. Ω Notation
  3. θ Notation
- This is also known as an algorithm's growth rate.
- Asymptotic Notations are used,
  1. To characterize the complexity of an algorithm.
  2. To compare the performance of two or more algorithms solving the same problem.

The Class P

- The class P consists of those problems that are solvable in polynomial time by deterministic algorithms.
  - More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k, where n is the size of the input to the problem.
  - For example, $O(n^3)$, $O(n^4)$, $O(logn)$, Fractional Knapsack, MST, Sorting algorithms etc…
  - P is a complexity class that represents the set of all decision problems that can be solved in polynomial time.
  - That is, given an instance of the problem, the answer yes or no can be decided in polynomial time.

Tractable/Intractable Problems

- Problems in P are also called **tractable**
- Problems **not** in P are **intractable** or **unsolvable**
  - Can be solved in reasonable time only for small inputs
  - Or, can not be solved at all

Example of Unsolvable Problem

- Turing discovered in the 1930's that there are problems **unsolvable** by any algorithm.
- The most famous of them is the **halting problem**
  - Given an arbitrary algorithm and its input, will that algorithm eventually halt, or will it continue forever in an "*infinite loop*?"

Optimization & Decision Problems

- **Decision problems**
  - Given an input and a question regarding a problem, determine if the answer is yes or no
- **Optimization problems**
  - Find a solution with the “best” value
- Optimization problems can be cast as decision problems that are easier to study
  - E.g.: Shortest path: G = unweighted directed graph
    - Find a path between u and v that uses the fewest edges
    - Does a path exist from u to v consisting of at most

Examples of Intractable Problems

Hamiltonian Paths

Optimization Problem: Given a graph, find a path that passes through every vertex exactly once

Decision Problem: Does a given graph have a Hamiltonian Path ?

Traveling Salesman

Optimization Problem: Find a minimum weight Hamiltonian Path

Decision Problem: Given a graph and an integer k, is there a Hamiltonian Path with a total weight at most k ?

Intractable Problems

- Can be classified in various categories based on their degree of difficulty
  - NP
  - NP-complete
  - NP-hard

Nondeterministic and NP Algorithms

Nondeterministic algorithm = two stage procedure:

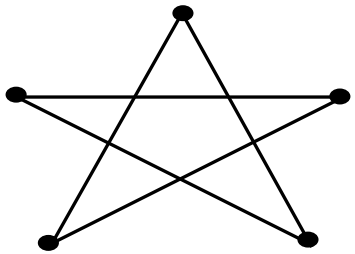
- Nondeterministic ("guessing") stage:
  - generate randomly an arbitrary string that can be thought of as a candidate solution ("certificate")
- Deterministic ("verification") stage:
  - take the certificate and the instance to the problem and returns YES if the certificate represents a solution

NP algorithms (Nondeterministic polynomial)

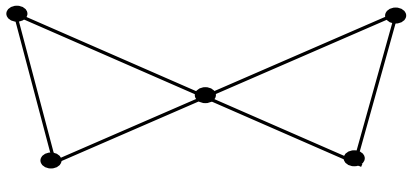
- verification stage is polynomial

E.g.: Hamiltonian Cycle

- **Given:** a directed graph G = (V, E), determine a simple cycle that contains each vertex in V
  - Each vertex can only be visited once
- **Certificate**:
  - Sequence: $<v_1, v_2, v_3, \ldots, v_{|V|}>$



hamiltonian



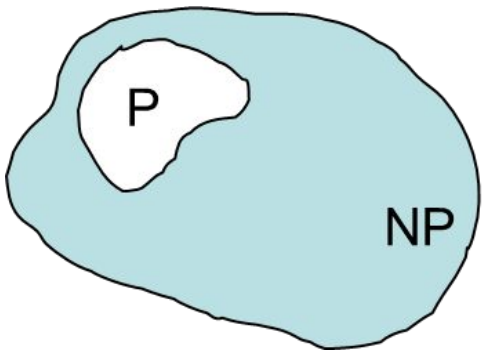
not hamiltonian

The NP class

- NP is Non-Deterministic polynomial time.
- The class NP consists of those problems that are verifiable in polynomial time.
- NP is the class of decision problems for which it is easy to check the correctness of a claimed answer, with the help of a little extra information.
- Hence, we are not asking for a way to find a solution, but only to verify that an alleged solution really is correct.
- Every problem in this class can be solved in exponential time using exhaustive search.

P and NP Class Problems

- P = set of problems that **can be solved** in polynomial time
  - NP = set of problems for which a solution **can be verified** in polynomial time
  - $P \subseteq NP$



Classification of NP Problems

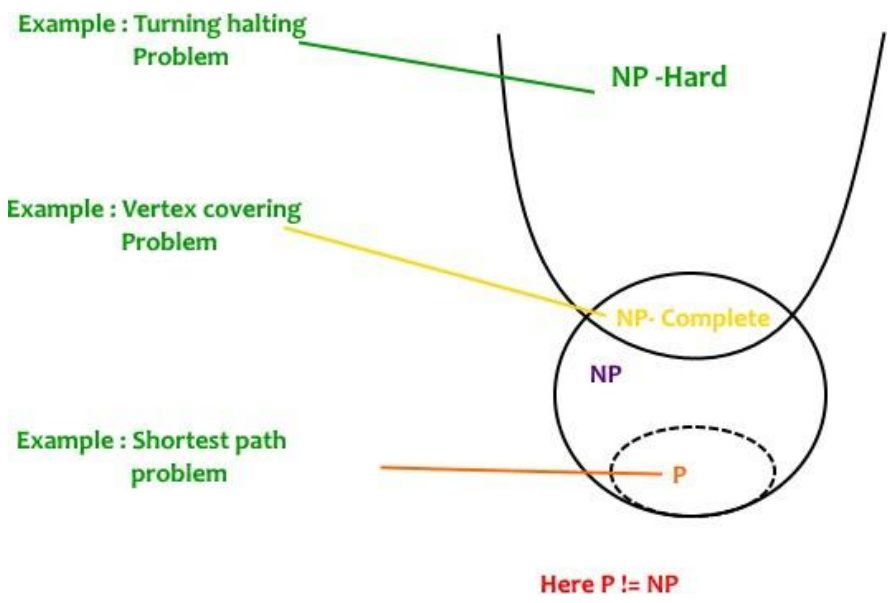
NP Complete

- NP-complete problems are a set of problems to each of which any other NP-problem can be reduced in polynomial time, and whose solution may still be verified in polynomial time.
- No polynomial-time algorithm has been discovered for an NP-Complete problem.
- *NP-Complete is a complexity class which represents the set of all problems X in NP for which it is possible to reduce any other NP problem Y to X in polynomial time.*

NP Hard

- NP-hard problems are those at least as hard as NP problems, i.e., all NP problems can be reduced (in polynomial time) to them.
- NP-hard problems need not be in NP, i.e., **they need not have solutions verifiable in polynomial time.**
- *The precise definition here is that a problem X is NP-hard, if there is an NP-complete problem Y, such that Y is reducible to X in polynomial time.*

P, NP Complete and NP Hard



Approximation Algorithm

- An optimization problem is the problem of finding the best solution from the set of all feasible solutions.
- The objective is a quantitative measure of system's performance that may be either minimized or maximized depending on the nature of a problem considered.
- A large number of optimization problems which are required to be solved in practice are NP- complete or NP-hard.
- For such problems, it is not possible to design algorithms that can find exactly optimal solution to all instances of the problem in polynomial time.
- But it may still be possible to find near-optimal solutions in polynomial time. In practice, near- optimality is often good enough.

Approximation Algorithm

- An algorithm that returns <u>near-optimal solutions</u> is called an approximation algorithm.
- Approximation Algorithm Definition: Given an optimization problem P, an algorithm A is said to be an approximation algorithm for P, if for any given instance I, it returns an approximate solution, that is a feasible solution.
- The goal of an approximation algorithm is to come as close as possible to the optimum value in a reasonable amount of time which is at the most polynomial time.

Thank
You!